```
def print_board(board):
    """Helper function to print the chessboard neatly."""
    for row in board:
        print(" ".join(row))
    print()


def is_safe(board, row, col, n):
    """Check if it's safe to place a queen at board[row][col]."""
    # 1. Check this row on the left side
    for i in range(col):
        if board[row][i] == "Q":
            return False
    # 2. Check this row on the right side
    for i in range(col + 1, n):
        if board[row][i] == "Q":
            return False
    # 3. Check this column (up and down)
    for i in range(n):
        if board[i][col] == "Q":
            return False
    # 4. Check upper diagonal to the left
    for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
        if board[i][j] == "Q":
            return False
    # 5. Check lower diagonal to the left
    for i, j in zip(range(row, n, 1), range(col, -1, -1)):
        if board[i][j] == "Q":
            return False

    # 6. Check upper diagonal to the right
    for i, j in zip(range(row, -1, -1), range(col, n, 1)):
```

```
        if board[i][j] == "Q":
            return False

    # 7. Check lower diagonal to the right
    for i, j in zip(range(row, n, 1), range(col, n, 1)):
        if board[i][j] == "Q":
            return False

    return True
def solve_n_queens_util(board, col, n, fixed_row):
    """Backtracking utility function to place the rest of the queens."""
    # Base Case: If all queens are placed, return True
    if col >= n:
        return True
 # If this column already has the user's fixed queen, skip to the next column
    if any(board[r][col] == "Q" for r in range(n)):
        return solve_n_queens_util(board, col + 1, n, fixed_row)

    # Try placing a queen in all rows for this column
    for row in range(n):
        # The user's row already has a queen, don't place another one in that row
        if row == fixed_row:
            continue

        if is_safe(board, row, col, n):
            board[row][col] = "Q"  # Place queen

            # Recur to place the rest of the queens
            if solve_n_queens_util(board, col + 1, n, fixed_row):
                return True
```

```
            board[row][col] = "."  # Backtrack (remove queen)

    return False
def solve_n_queens():
    # 1. Get board size
    n = int(input("Enter the size of the board (N): "))
    if n < 4 and n != 1:
        print(f"No solution exists for an {n}x{n} board.")
        return
    # Initialize empty board using '.'
    board = [["." for _ in range(n)] for _ in range(n)]
    # 2. Get the position of the first Queen from the user
    print(f"\nEnter the position for the 1st Queen (0 to {n-1}):")
    first_row = int(input(f"Enter Row: "))
    first_col = int(input(f"Enter Column: "))
    # Validate user inputs
    if not (0 <= first_row < n and 0 <= first_col < n):
        print("Invalid coordinates! Out of board boundaries.")
        return
    # Place the user's queen
    board[first_row][first_col] = "Q"

    print("\nInitial Board Layout:")
    print_board(board)
# 3. Solve for the remaining columns starting from column 0
    if solve_n_queens_util(board, 0, n, first_row):
        print("Final Solution Matrix:")
        print_board(board)
    else:
        print(
            f"No valid N-Queens solution exists with a Queen fixed at ({first_row}, {first_col})."
```

)

solve_n_queens()

The N-Queens Problem is a problem in which we have to place N queens on an N × N chessboard so that no two queens attack each other.A queen can attack another queen if they are in the same:Row (horizontal).Column (vertical).Diagonal.Objective: Place all N queens safely on the chessboard.For example, in the 4-Queens Problem, we have to place 4 queens on a 4 × 4 chessboard without attacking each other.

Backtracking is an algorithmic technique in which we try a possible solution, and if it is not valid, we go back and try another possibility.

What is the difference between recursion and backtracking?Recursion is a programming technique in which a function calls itself. Backtracking is an algorithmic technique that uses trial, checking, and undoing choices. Backtracking often uses recursion.

Pruning means stopping the exploration of a path as soon as we know it cannot produce a valid solution. For positive N, solutions exist for N = 1 and all N ≥ 4. There are no solutions for N = 2 or N = 3. What is the base condition in the program? When row == n, all queens have been placed successfully, so the function returns True

O(N!) is the commonly stated upper bound for the search. Space:$O(N^2)$ for the board, including O(N) recursion stack space.